J O S E P H I A N S C H O O L S

Real Backend Data Integration

Product Requirements Document: Exams and Report Cards Module

Prepared for: Deo Nsubuga, Academic Master

Author: Product team, with the PM Skills /write-prd workflow

Date: 4 November 2026

Status: Draft for review

Table of Contents

1. Executive Summary..... 1
2. Problem Statement..... 1
2.1 Who has this problem..... 1
2.2 Problem framing narrative (problem-statement skill)..... 1
2.3 Evidence..... 2
2.4 Final problem statement..... 2
3. Target Users and Personas..... 2
4. Strategic Context and Why Now..... 3
5. Solution Overview..... 4
5.1 What stays fixed..... 4
6. Success Metrics..... 4
7. User Stories and Requirements..... 5
7.1 Epic hypothesis..... 5
7.2 Story split (user-story and user-story-splitting skills)..... 5
7.3 Story S1: Model the academic domain..... 6
7.4 Story S2: Data access layer behind the existing seam..... 6
7.5 Story S3: Context from real selections..... 7
7.6 Story S4: Marks entry persists..... 7
7.7 Story S5: Reports and verification from real data..... 7
7.8 Story S6: Roles enforced server-side..... 8
8. Out of Scope..... 8
9. Dependencies and Risks..... 8
10. Open Questions..... 9
11. Self-Assessment..... 9

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

We are building a real data layer behind the existing Josephian Schools Exams and Report Cards module for the academic master, class teachers and parents, so that the school stops operating on demonstration data and starts using the module as its system of record. The module already delivers 23 working pages (exams administration, grading schemes, the full report-cycle workflow from marks entry to release, a parent portal and public verification) on top of a centralised demo configuration. The only missing layer is persistence: every mark, comment, approval and report card currently comes from deterministic demo fixtures.

This document defines the problem, the users, the scope of the data-integration release, measurable success criteria and implementation-ready stories. It was produced with the Product-Manager-Skills /prioritize and /write-prd workflows; the prioritization record that selected this initiative over parent-portal polish and analytics expansion is summarised in Section 4.

2. Problem Statement

2.1 Who has this problem

Deo Nsubuga, the academic master of Josephian Schools, is the owner of the reporting cycle. He configures exams, chases marks entry across nine class-subject deadlines per class, writes approvals and releases report cards to 12-plus classes and their parents. His class teachers enter marks in the module; his parents read the result through the portal or the printed sheet.

2.2 Problem framing narrative (problem-statement skill)

I am: the academic master of a growing Dar es Salaam school, responsible for every terminal report card the school issues, and the person teachers come to when marks or comments go missing.

Trying to: run one trusted reporting cycle per term in which marks are captured once, grades are computed consistently, approvals are recorded and parents receive the correct report the first time.

But: the module I approved shows Form 3A with twelve demo students and fabricated averages. Nothing I enter survives a restart that swaps the data source; none of it is the school's actual record; and I cannot hand parents a portal that displays someone else's child.

Because: the module was built against a demo fixture layer, and the school's real registers live outside the system, so there is no schema, no migration and no write path behind the screens.

Which makes me feel: that the impressive screens are a promise I cannot keep: every hour teachers spend entering real work into a demo is an hour I later have to apologise for.

2.3 Evidence

- The repository's own configuration documents the gap: src/lib/demo-config.ts states that its identifiers are temporary demonstration values and that Deo will connect real backend data later; every route is generated from that single seam.
- The persistence layer is scaffolded but empty of domain models: prisma/schema.prisma (SQLite) contains only the starter User and Post models, while src/lib/demo-data.ts generates students, marks, comments, deadlines and approvals in code.
- The parent portal and the public /verify page already render whatever the data layer returns, which makes real-data integration a correctness prerequisite before any parent-facing release, not an enhancement.

2.4 Final problem statement

The academic master needs a way to *run the reporting cycle on the school's real records* because the module currently serves deterministic demo data with no write path, which makes every entered mark disposable and blocks any parent-facing release.

3. Target Users and Personas

Proto-personas below follow the kit's proto-persona format. They are role-based and grounded in the module's own navigation model; they have not yet been validated by interviews, which is flagged in the self-assessment (Section 11).

Table 1: Proto-personas for the data-integration release

Persona	Role and context	Goals	Pains today	What changes for them
Academic-Master Deo (primary)	Owner of the reporting cycle; single module power user; comfortable with spreadsheets and keyboard-first data entry.	One trusted source of truth; cycle completes before term end; audit trail he can defend.	Cannot keep entered data; cannot release real reports; fears a parent seeing demo data.	Every screen reads and writes the school database; approvals and releases become real records.

Persona	Role and context	Goals	Pains today	What changes for them
Teacher Tumaini (secondary)	Class teacher for Form 3A; enters marks and comments per subject; low tolerance for new process.	Enter marks once, quickly, from the class register; see grades appear correctly.	Any entry today is throwaway; no confidence the sheet will persist.	Marks entry saves to the database with live grade feedback and drafts survive the session.
Parent Neema (tertiary)	Parent of a Form 3A student; checks the portal on a phone; non-technical.	See her child's correct terminal report and book a parents-day slot.	Portal currently shows demo students, so it is unusable as a source of truth.	Portal reads only her child's real records; verification codes resolve against real issues.

4. Strategic Context and Why Now

The /prioritize run scored three candidate initiatives for the coming term under the prioritization-advisor's recommendation (ICE scoring plus a value-versus-effort reading, chosen because the team is one developer and there is no production usage data for RICE reach estimates). Acquisition-channel and pricing steps of the workflow were assessed and recorded as not material for an internal school deployment.

Table 2: Prioritization record (ICE: impact x confidence x ease)

Initiative	Revenue connection (kit taxonomy)	I	C	E	ICE	Quadrant
Real backend data integration	Enabler: contract deliverable; strategic value (option 5 path)	3	85%	2	5.1	High value, high effort
Parent portal polish	Retention of parent engagement (option 2 path)	2	75%	4	6.0	High value, low effort
Analytics expansion	No direct revenue; strategic for leadership (option 5)	2	60%	3	3.6	Medium value, medium effort

The raw ICE ranking favours portal polish, but the workflow's own checkpoints flip the order, and two facts do. First, reversibility: data-model decisions are the only irreversible ones on the list, and they are the cheapest to get wrong expensively later. Second, dependency: the parent portal currently renders demo students to whoever opens it; polishing a portal that shows someone else's child to real parents is actively harmful, and analytics on demo data is equally hollow. Portal polish and analytics expansion are therefore blocked by

this initiative, not merely outranked by it. Confidence in the number-one ranking is high; the ranking would flip only if the school's existing registers turn out to be unavailable in digital form, in which case the fallback in Section 9 applies.

5. Solution Overview

We are not rebuilding screens. The 23 pages, their routes and the design system stay exactly as they are. We are replacing what sits behind them: a real data layer for the academic domain, exposed through the same function signatures the pages already call, so the demo fixture layer becomes a swappable implementation behind one seam.

The work has four layers. One: model the domain in Prisma (academic years, report cycles, classes, students, subjects, teaching assignments, marks, grading schemes, comments, approvals, batches, release records, verification codes, parents-day slots and holds), and migrate the SQLite database. Two: implement a data-access layer whose exported functions mirror today's demo-data API, so imports change once instead of everywhere. Three: switch the navigation context from the hardcoded demo identifiers (cycle cy1, class 01a1a064-..., student st1 in src/lib/demo-config.ts) to real selected records, keeping the resolveHref template mechanism unchanged. Four: add authentication and role gates so academic master, class teacher and parent see the permissions they already have in the UI, enforced server-side this time.

Demo mode is retained as an explicit configuration switch for training and sales demonstrations; it is never the default and never serves real students. This keeps the documented promise in demo-config.ts: replacing the demo values with real identifiers requires no redesign of the navigation.

5.1 What stays fixed

- All 23 routes and their URL shapes, including dynamic segments for cycle, class and student.
- The design system: burgundy token theme, collapsible sidebar, and the passing ux-ui-agent-skills gate suite (zero hardcoded values, WCAG-passing token source, no overflow at 390 and 280 pixels).
- Business logic already built: grading computation, readiness checks, review and approval flow, batch generation, release management and verification checksums.

6. Success Metrics

Success is defined by data truthfulness first, then by preserved behaviour, then by performance. Metrics are measurable from the repository and the running system, not from surveys.

Table 3: Success metrics for the release

Tier	Metric	Current	Target	Measured by
Primary	Share of module pages serving real school data	0% (demo fixtures)	100% for all 23 pages by term end	Code audit: no page imports demo-data at release
Secondary	Marks entry save latency (per cell commit)	Not applicable (no writes)	Under 1 second, 95th percentile locally	Instrumented save handler
Secondary	Report-card batch generation, one class of 12	Instant (fixtures)	Under 30 seconds end to end	Batch run timing log
Secondary	Public verification of an issued code	Demo codes only	Every issued code resolves with correct checksum	Verify flow test
Guardrail	Route stability	23 routes	23 routes, zero URL changes	Link and navigation audit
Guardrail	Design gates	All green	Stay green after the change	Kit gate suite rerun
Guardrail	Demo mode	Default	Opt-in only, flagged screens, no real records	Config and code review

7. User Stories and Requirements

7.1 Epic hypothesis

We believe that connecting the existing 23-page module to a real school database behind the current demo configuration seam will make the module the school's system of record this term, because the screens, workflows and business logic are already built and the only missing layer is persistence. We will measure success by the primary metric in Section 6 within 30 days of cutover.

7.2 Story split (user-story and user-story-splitting skills)

The epic splits along Pattern 1 (workflow steps) and Pattern 6 (external dependency boundary): the schema must exist before the access layer; the access layer before the context switch; the context switch before role enforcement. Each story below ships user-visible value on its own slice.

Table 4: Story map for the epic

Story	Slice (kit split pattern)	Depends on
S1 Domain schema	Data foundation (Pattern 5: major effort milestone)	-
S2 Data access layer	Same signatures behind the seam (Pattern 6: boundary)	S1
S3 Context switch	Navigation context from real records (Pattern 1: workflow step)	S2
S4 Marks entry persistence	First real write path (Pattern 1: workflow step)	S2, S3
S5 Reports and verification	Generation and codes from real data (Pattern 1)	S4
S6 Roles and access	Server-side enforcement (Pattern 2: business rule variations)	S3

7.3 Story S1: Model the academic domain

As *the academic master*, I want the school's classes, students, subjects, cycles and marks stored in the module's database, so that the data I see is the data the school keeps.

Acceptance criteria (Gherkin):

- Scenario: schema covers the module. Given the migrated database, when the access layer queries each domain entity used by the 23 pages, then every query resolves without falling back to demo fixtures.
- Scenario: a term is representable. Given a new academic year, when I create a report cycle for Term 1 with start and end dates, then the cycle appears in the cycle selector on every context-aware page.
- Scenario: grading scheme is data. Given a grading scheme with boundaries A to F, when marks are stored, then boundaries are read from the database, not from code constants, and changing a boundary changes computed grades on recompute.

7.4 Story S2: Data access layer behind the existing seam

As *the developer*, I want a data-access layer whose exported functions mirror the demo-data API, so that pages switch source without route or markup changes.

Acceptance criteria:

- Scenario: signature parity. Given the function list exported by src/lib/demo-data.ts, when the access layer is implemented, then every consumer compiles against the new module with no signature changes.

- Scenario: demo switch. Given the configuration flag, when demo mode is enabled, then pages render fixtures exactly as today; when disabled, the same pages resolve only real records.
- Scenario: no silent demo. Given demo mode disabled, when a query returns no record, then the UI shows an empty state, never a demo row.

7.5 Story S3: Context from real selections

As *the academic master*, I want the cycle, class and student context to come from my actual selections, so that navigating the module always shows the records I chose.

Acceptance criteria:

- Scenario: selector drives navigation. Given the header cycle and class selectors, when I pick Term 1 2026 and Form 3B, then every context-aware link resolves to that cycle and class via the existing resolveHref templates.
- Scenario: deep links keep working. Given a bookmarked URL with real identifiers, when I open it directly, then the page resolves that record; unknown identifiers render the existing empty state instead of crashing.

7.6 Story S4: Marks entry persists

As *a class teacher*, I want my marks saved as I type and survive the session, so that entering a class of results is work done once.

Acceptance criteria:

- Scenario: save and recompute. Given marks entry for Mathematics in Form 3A, when I commit a mark of 91 for a student, then the value persists, the grade pill recomputes from the scheme and the class statistics update.
- Scenario: draft survival. Given unsaved edits, when the page is reloaded or the session drops, then committed marks persist and uncommitted drafts are recoverable or clearly discarded, never silently lost.
- Scenario: concurrent entry. Given two teachers entering the same class-subject, when both commit, then the second write is rejected with a clear message instead of overwriting the first.

7.7 Story S5: Reports and verification from real data

As *the academic master*, I want report cards, batches, release and verification codes computed from real records, so that what a parent receives is defensible.

Acceptance criteria:

- Scenario: print matches data. Given a student with real marks, when I preview or print the report card, then every figure on the sheet equals the stored record and the sheet shows the real verification code.
- Scenario: verification resolves. Given an issued report code, when anyone enters it on /verify, then the page confirms issue date and checksum from the database; a fabricated code returns not found.

7.8 Story S6: Roles enforced server-side

As *a parent*, I want the portal to show only my child's records, so that the school's data stays private.

Acceptance criteria:

- Scenario: parent scope. Given a signed-in parent, when any portal API is called, then only records for that parent's children are returned, regardless of client-side navigation.
- Scenario: teacher scope. Given a class teacher, when marks entry opens, then writable subjects are the ones assigned to that teacher; the academic master retains cycle-wide rights.
- Scenario: demo protection. Given demo mode enabled on a training instance, when a user attempts a write, then the write is refused or sandboxed and no real record is touched.

8. Out of Scope

The release deliberately excludes the following, each recorded with its reason so scope debates end here rather than in the sprint.

- Parent portal polish and visual changes: blocked-by design; scheduled immediately after this release and before the parent-facing announcement.
- Analytics expansion beyond today's dashboard: deferred until leadership names the specific reports they act on; no demand signal exists yet.
- SMS or email notification delivery: the notification feed stays in-app; delivery channels need a provider decision that is out of scope.
- Multi-school tenancy: the school operates as one deployment; the schema notes the extension point but does not implement it (see Section 10).
- Mobile-native applications: the module is responsive and stays web-only.

9. Dependencies and Risks

Table 5: Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Existing school registers are not available in digital, importable form	Medium	High	One-day spike before sprint 1 maps register sources; fallback is a one-off assisted import (CSV per class) plus manual entry through the module, re-sequenced before S4.
Schema drift against the types the UI already consumes	Medium	Medium	The access layer owns translation; types.ts remains the contract; a mismatch fails compile, not runtime.
Authentication scope creep delaying go-live	High	Medium	S6 ships with the platform's existing session mechanism and three roles only; password reset and directory integration are follow-ups.
Single-developer bus factor across a term-long epic	Medium	Medium	Stories are sequenced to ship value independently; schema and access layer land first so later work is additive.
SQLite ceiling under school-wide concurrent writes	Low	Low	Access layer isolates the engine; the documented Postgres migration path stays available without page changes.

Dependencies: design is not blocking (no visual change); the module's own business logic and tests provide the verification harness; the school register spike (above) is the only external dependency.

10. Open Questions

- [Open Question] Which system currently holds the authoritative class registers, and can it export per-class CSV? Determines whether the fallback import in Section 9 is needed.
- [Open Question] Is single-school tenancy a permanent product decision, or must the schema carry a school identifier from day one for a future multi-campus rollout?
- [Open Question] Do grading schemes ever change mid-cycle, and if so must historical report cards preserve the scheme in force at issue time?
- [Open Question] What are the school's backup and retention expectations for student records, and who owns them?
- [Assumption] The term deadline forces go-live before the next reporting cycle; if the calendar slips, portal polish re-enters the term's scope first.

11. Self-Assessment

Strongest section: the solution overview and its seam strategy, because it is grounded in the repository's actual structure (centralised demo configuration, function-signature parity, route templates) rather than in aspiration; the guardrail metrics make regression observable.

Weakest section: the personas. They are role-based proto-personas derived from the module's navigation model and the configuration file; no teacher or parent interview has been conducted yet. The risk is mis-sized effort on marks-entry ergonomics.

Top assumptions to validate: (1) the school's registers are digitally importable within the term; (2) single-school tenancy holds for at least two years; (3) the go-live deadline precedes the next reporting cycle.

Recommended next step: run the one-day schema-mapping spike (register sources versus the domain model in Story S1) and one 30-minute marks-entry shadowing session with a class teacher, then re-confirm this document before sprint 1.